Air Quality Prediction System

Project Id: AIML-15

Name : Harsh Raunchela**(Team Leader)**
Class Roll Number: 47
University Roll Number :2028723
Section and Group: K – G2

Name : Devansh Dangwal
Class Roll Number:45
University Roll Number :2028700
Section and Group:K-G2

Name : Piyush Maurya
Class Roll Number:62
University Roll Number : 2028797
Section and Group: K-G1

TABLE OF CONTENTS

1. Problem Description

The Air Quality Prediction System is a Python-based machine learning application developed to automate the analysis and prediction of outdoor air quality levels based on pollutant concentration and meteorological data. Using core concepts of Machine Learning such as data preprocessing, feature selection, classification algorithms, model evaluation, and visualization, this project predicts the Air Quality Index (AQI) category for a given set of environmental readings.

The system classifies air quality into the following six categories as per Indian AQI standards:

Category	PM2.5 Range (µg/m³)	Health Implication
Good	0 – 30	Minimal impact
Satisfactory	31 – 60	Minor breathing discomfort
Moderate	61 – 90	Discomfort for sensitive people
Poor	91 – 120	Breathing discomfort for most
Very Poor	121 – 250	Respiratory illness on prolonged exposure
Severe	> 250	Serious health effects for all

2. Architecture

The system follows a complete end-to-end machine learning pipeline as described below:

Step	Stage	Description
1	**Raw Data Input**	Load air quality dataset with pollutant and meteorological readings (8784 rows, 25 columns)
2	**Data Preprocessing**	Handle missing values, drop empty columns, extract datetime features (Month, Hour, Weekday)
3	**AQI Label Creation**	Create target variable (6 AQI categories) based on PM2.5 values using Indian standards
4	**Feature Selection**	Correlation analysis to remove redundant features + SelectKBest to remove irrelevant ones
5	**Train/Test Split**	80% training, 20% testing split using train_test_split
6	**Model Training**	Train Logistic Regression, KNN, and SVM classifiers
7	**Hyperparameter Tuning**	GridSearchCV with 5-fold cross validation to find optimal parameters
8	**Model Evaluation**	Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC Curve
9	**Prediction**	Best model (SVM) used for final AQI category prediction

3. Models Used(Applied/ Planned) Give a brief introduction of the models used

3.1 Logistic Regression

Logistic Regression is a classification algorithm that models the probability of each class using the sigmoid function. Despite its name, it is used for classification tasks. It works by finding the best decision boundary that separates classes.

- Type: Linear Classifier
- Best Parameters found: C = 10, solver = lbfgs
- Training Accuracy: 97.0% | Testing Accuracy: 97.0%

3.2 K-Nearest Neighbors (KNN)

KNN is a non-parametric algorithm that classifies a data point based on the majority class among its K nearest neighbors in the feature space. Distance metrics are used to find the nearest neighbors.

- Type: Instance-based Classifier
- Best Parameters found: K = 11, metric = manhattan, weights = uniform
- Training Accuracy: 83.1% | Testing Accuracy: 78.4%

3.3 Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximizes the margin between classes. It is one of the most powerful classifiers for high-dimensional data and works well with both linear and non-linear boundaries using kernels.

- Type: Margin-based Classifier
- Best Parameters found: C = 10, kernel = linear, gamma = scale
- Training Accuracy: 98.9% | Testing Accuracy: 98.6%
-

Model	Train Accuracy	Test Accuracy	F1 Score
Logistic Regression	97.0%	97.0%	0.97
KNN	83.1%	78.4%	0.77
SVM (Best)	**98.9%**	**98.6%**	**0.98**

-

4. Evaluation metrics used

Metric	Formula	Description
Accuracy	TP + TN / (TP + TN + FP + FN)	Overall correct predictions out of all predictions
Precision	TP / (TP + FP)	Of all predicted positives, how many are actually positive
Recall	TP / (TP + FN)	Of all actual positives, how many were correctly predicted
F1 Score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of Precision and Recall
Confusion Matrix	N/A	Matrix showing correct and incorrect predictions per class
ROC Curve / AUC	Area under TPR vs FPR curve	Measures model's ability to distinguish between classes

5. Preliminary Results and Visualizations

The following visualizations were generated during the project. Please insert your actual screenshots from your Jupyter Notebook in the spaces indicated below.

5.1 Feature Correlation Heatmap

The heatmap below shows the correlation between all features. Dark red indicates high positive correlation (redundant features), dark blue indicates negative correlation.

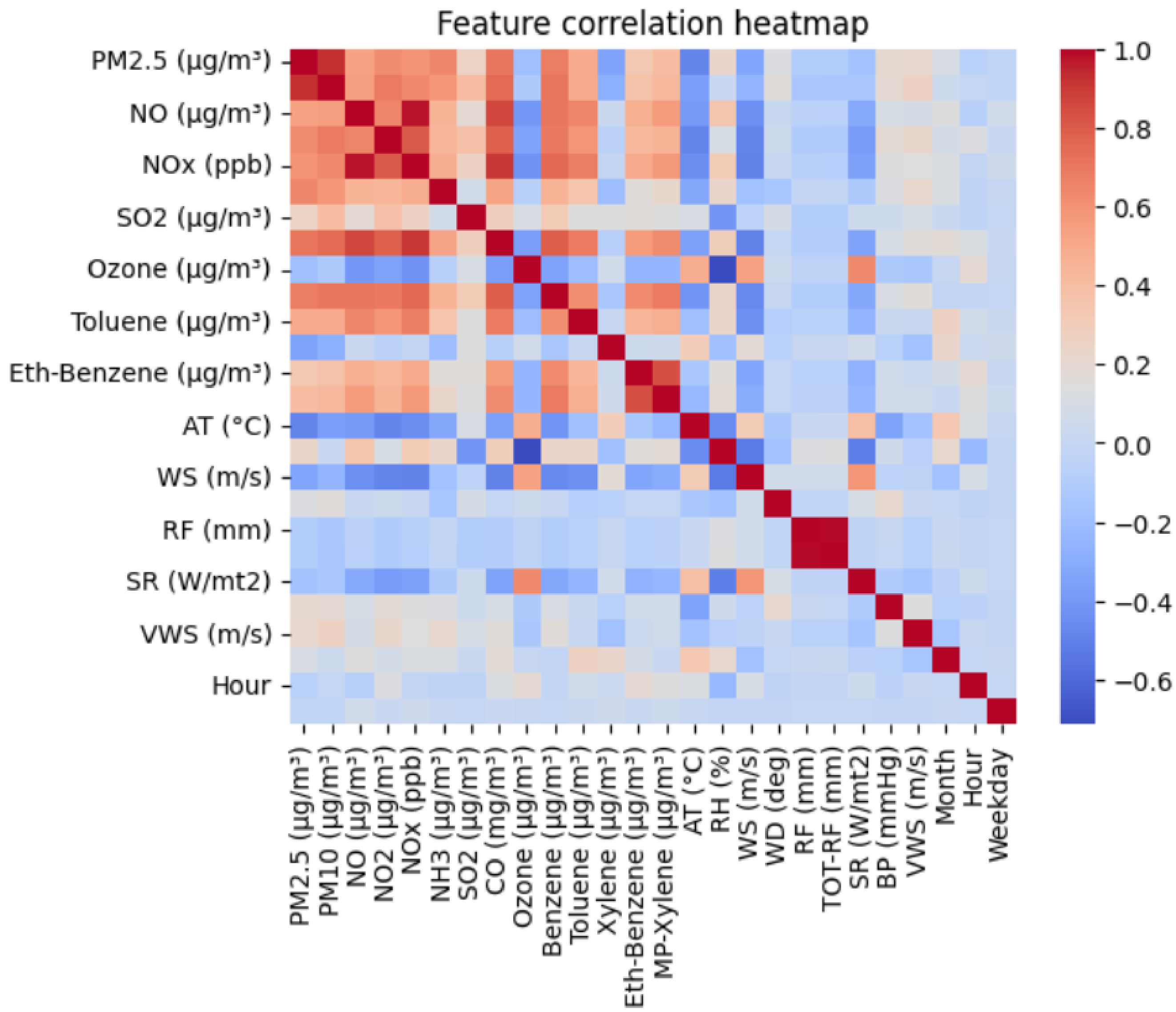

Fig. 1 — Feature Correlation Heatmap

5.2 Model Comparison Chart

The bar chart below compares the Accuracy and F1 Score of all three trained models.

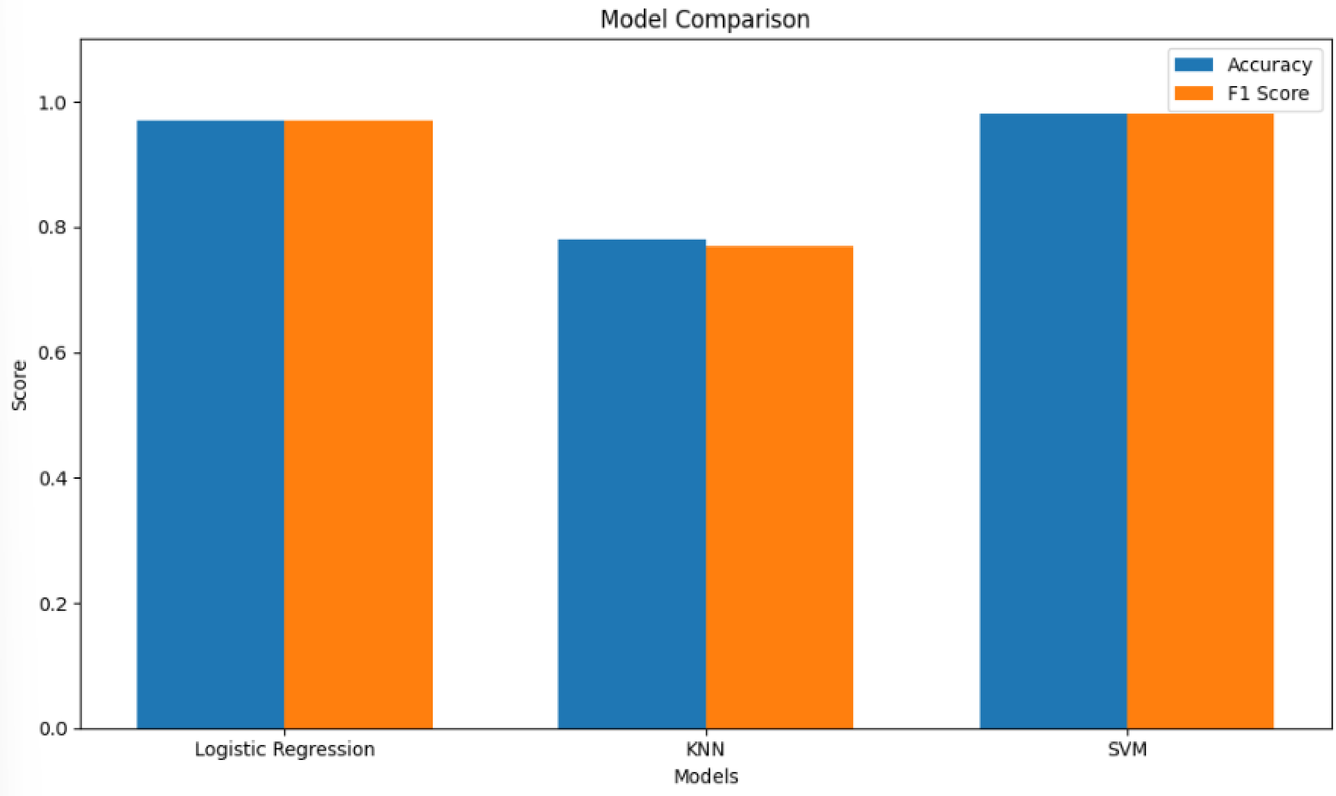

Fig. 2 — Model Comparison Chart (Accuracy & F1 Score)

5.3 Confusion Matrix (Best Model — SVM)

The confusion matrix shows predictions vs actual labels for the SVM model. Values on the diagonal represent correct predictions.

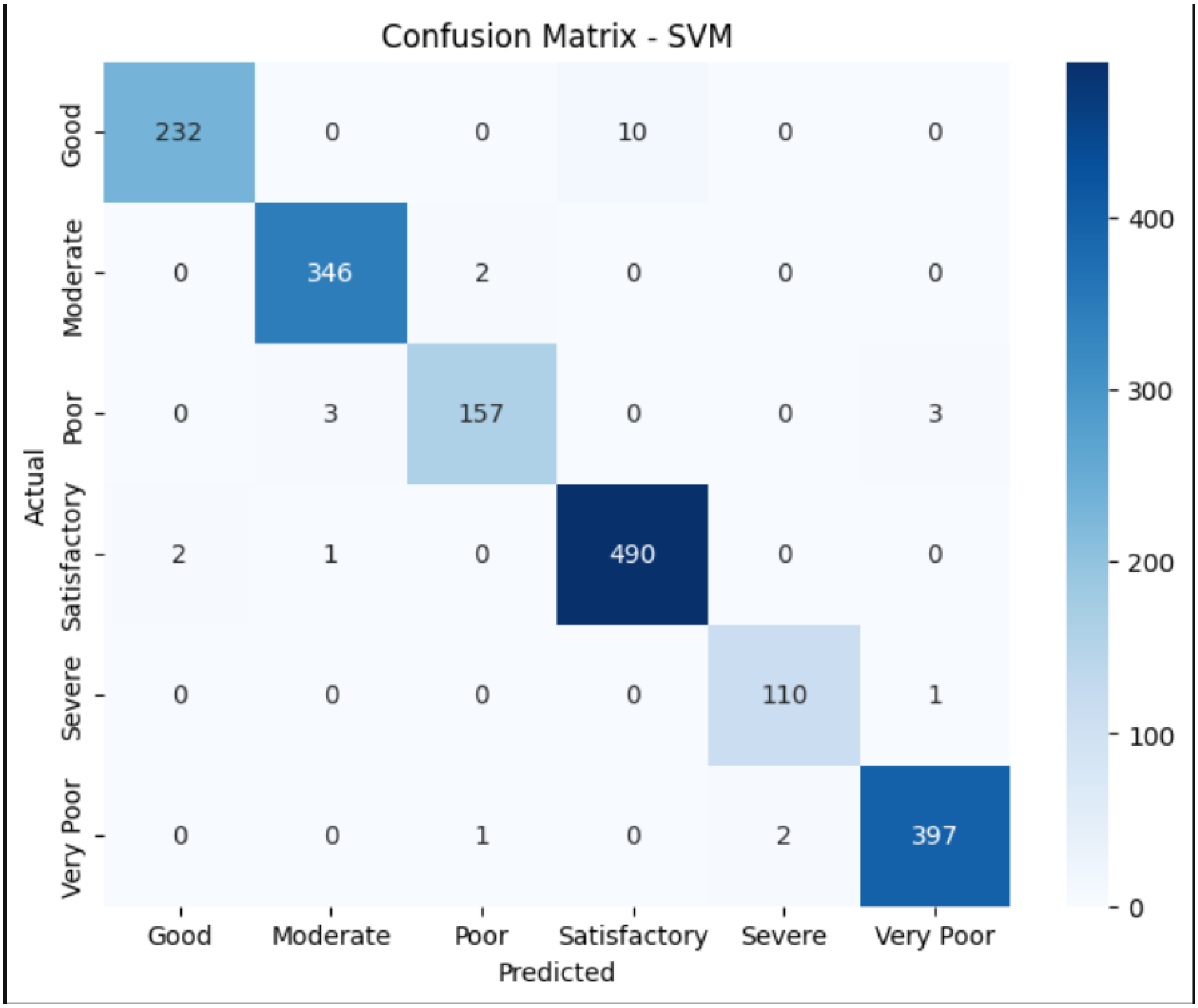

Fig. 3 — Confusion Matrix (SVM)

5.3 ROC CURVE (Best Model — SVM)

The ROC Curve is a graph that shows how well a classification model distinguishes between classes by plotting True Positive Rate against False Positive Rate at different thresholds.

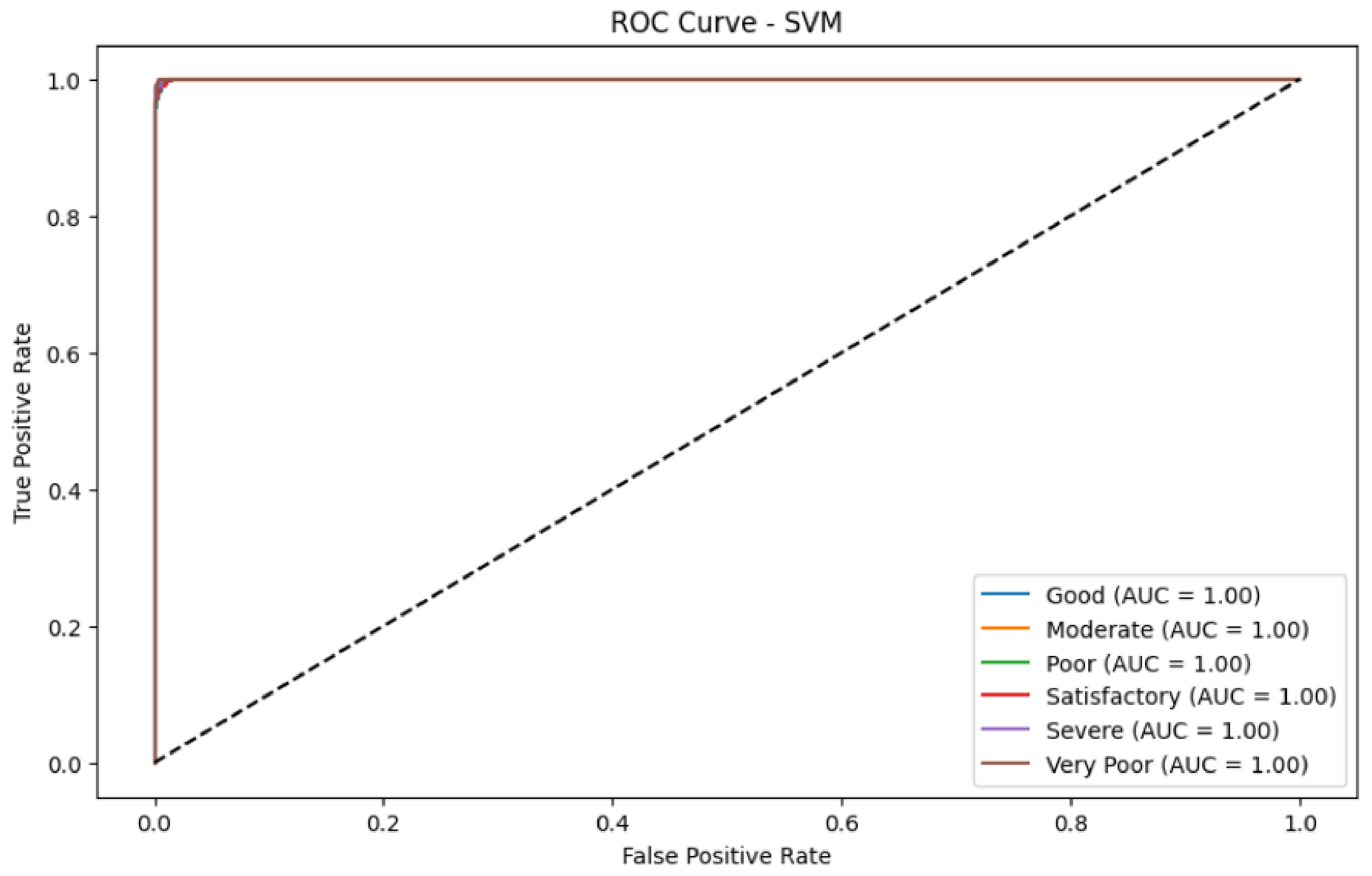

Fig. 4 — ROC Curve (SVM)

6. References:

Books:

- Let Us Python – Yashavant Kanetkar
- Hands-On Machine Learning with Scikit-Learn – Aurelien Geron

Online Resources:

- Scikit-learn Documentation – https://scikit-learn.org/
- GeeksforGeeks – https://www.geeksforgeeks.org/
- Kaggle Air Quality Datasets – https://www.kaggle.com/
- Stack Overflow – https://stackoverflow.com
- TutorialsPoint – https://www.tutorialspoint.com